

POS Kiosk Backend — Technical Manual

NestJS architecture, environment, database, API reference for every module, and Windows deployment.

Written for

Backend Developers & DevOps

Contents

1. Architecture
2. Prerequisites & install
3. Environment variables
4. Commands reference
5. Authentication flow
6. API reference — catalogue & menu
7. API reference — selling
8. API reference — users, inventory, reports
9. Health, ERP sync & cron
10. Device transfer internals
11. Migrations & schema
12. Production build & Windows installer
13. Verification & troubleshooting

Section numbers above match the numbered banners in the body. Use your PDF reader's bookmarks / outline panel to jump between sections.

NOTE The backend lives in the `be/` folder. Node.js 18+ (22 LTS for installer builds) and Docker are assumed. `npm install` must be run with `--legacy-peer-deps`.

1. Architecture

NestJS (TypeScript) REST API over PostgreSQL, using TypeORM. One process, deployed either via Docker (dev) or as a Node SEA Windows service (production installer).

1.1 Module layout

Each domain lives under `be/src/<domain>/` :

```
products/  
  dto/           input/output DTOs (class-validator)  
  entities/      TypeORM entities  
  mapper/        entity <=> DTO mappers (pure static classes)  
  services/      one file per use-case (CreateProductService, ...)  
products.controller.ts  
products.service.ts  orchestrates the feature services  
products.module.ts
```

1.2 Cross-cutting

Concern	Mechanism
Auth	JWT via Passport. Global JWT guard; <code>@Public()</code> opts out. <code>@CurrentUser()</code> injects identity. Role guards: <code>@RequireAdminOrSupervisor()</code> , <code>AdminOrSupervisorGuard</code> , <code>@RequireAdminOrSupervisorOrSelf('id')</code> .
Events	<code>EventEmitterModule</code> (in-process).
Scheduling	<code>ScheduleModule</code> + <code>CronJobsModule</code> .
Validation	<code>class-validator</code> / <code>class-transformer</code> on DTOs, global <code>ValidationPipe</code> .
API docs	Swagger at <code>/api/docs</code> .
Global prefix	<code>api/v1</code> (health also mounted under it).

1.3 Data access rules

- TypeORM with PostgreSQL. **Never** `synchronize: true`.
- Schema changes only via migrations in `src/database/migrations/` (timestamp-named).
- `src/database/migrations/index.ts` must stay in sync — `npm run migration:sync-index` (also run automatically by `build:sea`).
- Seeders in `src/database/seeders/`; their index is likewise synced.

2. Prerequisites & install

Tool	Version
Node.js	18+ for dev; 22 LTS to run <code>build:sea</code> for the installer
npm	bundled with Node
Docker + Compose	any recent (manages Postgres + app)
PostgreSQL	16 (via Docker in dev; portable binaries bundled in the installer)

```
cd be
cp .env.example .env          # then edit
npm install --legacy-peer-deps # REQUIRED flag
npm run docker:build:up       # build image + start Postgres & app
npm run migration:up
npm run seed:run
# API:      http://localhost:3000/api/v1
# Swagger:  http://localhost:3000/api/docs
```

Without Docker

```
docker compose up -d postgres # or a local Postgres 16
npm install --legacy-peer-deps
npm run migration:up
npm run db:reset              # wipe + re-run every seeder
npm run seed:run
npm run start:dev
```

IMPORTANT `npm run lint` is `eslint --fix` across the whole tree — it rewrites unrelated files. Scope lint to changed files and use `npm run build` for a full type-check.

3. Environment variables

Variable	Default / example	Purpose
NODE_ENV	development	Runtime mode
PORT	3000	HTTP port
POSTGRES_HOST	localhost	DB host
POSTGRES_PORT	5432	DB port
POSTGRES_USER	postgres	DB user (needs superuser for device-transfer import)
POSTGRES_PASSWORD	postgres	DB password — set a strong one in prod
POSTGRES_DB	pos_db	Database name
JWT_SECRET	random hex	Access-token signing key
JWT_EXPIRES_IN	15m	Access-token lifetime
JWT_REFRESH_SECRET	random hex	Refresh-token signing key (separate from above)
JWT_REFRESH_EXPIRES_IN	7d	Refresh-token lifetime
KIOSK_NO	1	Kiosk number — dev only ; in prod it is read from C:\POSKiosk\settings.txt
ERP_SYNC_ENABLED	true	Enable the back-office integration
ERP_BASE_URL	http://localhost:5000	ERP base URL
ERP_USERNAME / ERP_PASSWORD	—	ERP credentials
ERP_TERMINAL_ID	POS-1	Terminal id reported to the ERP
ERP_STORE_CODE	MAIN	Store code reported to the ERP

NOTE The installer bundles `.env.prod` as the runtime `.env`. Generate JWT secrets with `openssl rand -hex 48`. Never commit `.env*`.

4. Commands reference

Command	Does
<code>npm run start:dev</code>	Dev server with hot reload
<code>npm run build</code>	Full <code>nest build</code> / type-check
<code>npm run lint</code> / <code>format</code> / <code>format:check</code>	ESLint --fix (whole tree) / Prettier
<code>npm run test</code> / <code>test:watch</code> / <code>test:e2e</code> / <code>test:cov</code>	Unit / watch / e2e / coverage
<code>npx jest --testPathPattern=src/products/products.service.spec.ts</code>	Single test file
<code>npm run migration:up</code> / <code>:down</code> / <code>:show</code>	Apply / revert last / list status
<code>npm run migration:generate</code> / <code>:create</code>	Generate from entity diff / blank file
<code>npm run migration:sync-index</code>	Rebuild <code>migrations/index.ts</code>
<code>npm run seed:run</code> / <code>seed:create</code>	Run all seeders / scaffold one
<code>npm run db:reset</code>	Drop + recreate (dev only)
<code>npm run docker:build:up</code> / <code>docker:dev</code> / <code>docker:down</code> / <code>docker:logs</code>	Container lifecycle
<code>npm run pm2:start</code> / <code>:restart</code> / <code>:logs</code> / <code>:monit</code>	PM2 process management (alt. production runner)
<code>npm run build:sea</code>	Sync indexes + nest build + package to <code>be/POSBackend.exe</code> (Node SEA)
<code>npm run check:quotes</code> / <code>fix:quotes</code>	Detect / repair curly quotes in <code>installer.iss</code>

5. Authentication flow

Endpoint	Body	Returns
POST /api/v1/auth/login/email	email + password (LocalAuthGuard)	access + refresh tokens
POST /api/v1/auth/login/user-id	userId + password	tokens
POST /api/v1/auth/login/pin	userId + 6-digit PIN (kiosk login)	tokens + isPinChanged flag
POST /api/v1/auth/refresh	refresh token	new token pair
GET /api/v1/auth/me	Bearer token	current user

The kiosk uses **PIN login**; a `false isPinChanged` forces the client through the PIN-setup screen (`PATCH /users/:id`). `POST /users/verify-pin` backs the on-the-spot supervisor authorisation for voids, refunds and report closes. `GET /users/roster` is `@Public()` and feeds the kiosk's "Who's working?" screen.

6. API reference — catalogue & menu

Products — `<code>/api/v1/products</code>`

Method	Path	Summary
POST	<code>/products</code>	Create a product
POST	<code>/products/import-csv</code>	Import categories/products/variants from a CSV (upsert or replace)
GET	<code>/products</code>	List all products
GET	<code>/products/:id</code>	Get a product by id (kiosk product-details contract)
PATCH	<code>/products/:id</code>	Update a product

Product Variants — `<code>/api/v1/product-variants</code>`

Method	Path	Summary
POST	<code>/product-variants</code>	Create a variant (admin/supervisor)
GET	<code>/product-variants/names</code>	Distinct variant names used across products
GET	<code>/product-variants/by-product/:productId</code>	Variants for a product
GET	<code>/product-variants/:id</code>	Get a variant
PATCH	<code>/product-variants/:id</code>	Update a variant (admin/supervisor)

Product Groups — `<code>/api/v1/product-groups</code>`

Method	Path	Summary
POST	<code>/product-groups</code>	Create a group (multipart, optional <code>image</code>)
GET	<code>/product-groups</code>	List all groups
GET	<code>/product-groups/:id</code>	Get a group
GET	<code>/product-groups/:id/products</code>	Products in a group (paginated)
PATCH	<code>/product-groups/:id</code>	Update a group (multipart)
DELETE	<code>/product-groups/:id</code>	Remove a group

Modifier Groups — `<code>/api/v1/modifier-groups</code>`

Method	Path	Summary
POST	/modifier-groups	Create a group with options
GET	/modifier-groups · /:id	List / get
PATCH	/modifier-groups/:id	Update
DELETE	/modifier-groups/:id	Remove

Catalog (kiosk read path) — `/api/v1/catalog`

Method	Path	Summary
GET	/catalog/categories	Active categories + available product counts
GET	/catalog/products	Available products with nested category + modifier groups (raw SQL, fast)
GET	/catalog/modifier-groups	All modifier groups with nested modifiers
GET	/catalog/admin/categories	All categories incl. inactive (authed)
GET	/catalog/admin/products	All products incl. disabled/unavailable (authed)
POST	/catalog/admin/categories	Create category (admin/supervisor)
PATCH	/catalog/admin/categories/:id	Update category (admin/supervisor)

NOTE GET /catalog/products and GET /products/:id use different data paths and must be kept consistent. Migration 1779582100000 dropped the catalog_-prefixed tables; migration 1779582000000 changed products.image_url BYTEA→TEXT and added price, is_available, sort_order, category plus the product_modifier_groups junction. Keep the Product entity and FindProductDetailsService in sync.

Menus / Store Menus / Tax / Currency / UOM

Base	Endpoints
/api/v1/menus	CRUD (POST , GET , GET/:id , PATCH/:id , DELETE/:id)
/api/v1/store-menus	CRUD, plus POST GET PATCH DELETE /store-menus/menu-items[/:id] against the default store menu
/api/v1/tax-categories	CRUD
/api/v1/currencies	CRUD
/api/v1/uom	CRUD

7. API reference — selling

Sales Orders — `<code>/api/v1/sales-orders</code>`

Method	Path	Summary
POST	<code>/sales-orders</code>	Create a sales order
GET	<code>/sales-orders</code>	List (paged/sortable — Transactions screen)
GET	<code>/sales-orders/current</code>	The current (open) order
GET	<code>/sales-orders/:id</code> · <code>/:id/with-items</code>	Order header / order with line items
GET	<code>/sales-orders/:id/item/:itemId</code>	One line item
PATCH	<code>/sales-orders/:id/item/:itemId</code>	Update a line item
DELETE	<code>/sales-orders/:id/item/:itemId</code>	Remove a line item
PATCH	<code>/sales-orders/:id/discount</code>	Apply a discount to the order
PATCH	<code>/sales-orders/:id/confirm</code>	Confirm / complete the sale
PATCH	<code>/sales-orders/:id/void</code>	Void — requires admin or supervisor PIN

Payments — `<code>/api/v1/payments</code>`, Refunds — `<code>/api/v1/refunds</code>`, Discounts — `<code>/api/v1/discounts</code>`

Base	Endpoints
<code>/payments</code>	POST , GET , GET/:id , PATCH/:id , DELETE/:id
<code>/refunds</code>	POST (with reason + authoriser), GET , GET/:id , PATCH/:id , DELETE/:id
<code>/discounts</code>	POST , GET , GET/:id , PATCH/:id , DELETE/:id

POS Terminals — `<code>/api/v1/pos-terminals</code>`

Method	Path	Summary
GET	/pos-terminals/my-terminal	Terminal assigned to the current user
PATCH	/pos-terminals/my-terminal	Update terminal details (legal name, address, TIN)
POST	/pos-terminals/register	Register a new terminal for the current admin
POST	/pos-terminals/my-terminal/payment-methods	Add a payment method
PATCH	/pos-terminals/my-terminal/payment-methods/:id	Update a payment method
DELETE	/pos-terminals/my-terminal/payment-methods/:id	Remove a payment method

NOTE The kiosk number in every sales-order number (e.g. SO-001-2026-0001) comes from KIOSK_NO in dev and C:\POSKiosk\settings.txt in production.

8. API reference — users, inventory, reports

Users — `/api/v1/users`

Method	Path	Summary / guard
POST	<code>/users</code>	Create — <code>@RequireAdminOrSupervisor()</code>
GET	<code>/users</code>	List — <code>@RequireAdminOrSupervisor()</code>
GET	<code>/users/authorizers</code>	Supervisors/admins who can authorise
GET	<code>/users/roster</code>	Public login roster (kiosk)
POST	<code>/users/verify-pin</code>	Verify a PIN (on-the-spot authorisation)
GET	<code>/users/:id</code>	Get a user
PATCH	<code>/users/:id</code>	Update — <code>@RequireAdminOrSupervisorOrSelf('id')</code> (also the PIN-change path)
DELETE	<code>/users/:id</code>	Delete — <code>@RequireAdminOrSupervisor()</code>

Supporting: `/api/v1/user-groups` , `/user-permissions` , `/user-details` , `/user-addresses` — each standard CRUD.

Inventory — materials, recipes, stock, counts

Base	Endpoints
<code>/api/v1/materials</code>	CRUD (<code>POST</code> , <code>GET</code> , <code>GET/:id</code> , <code>PATCH/:id</code> , <code>DELETE/:id</code>)
<code>/api/v1/material-types</code>	CRUD
<code>/api/v1/recipes</code>	CRUD — material components + quantities per product
<code>/api/v1/inventory-stocks</code>	On-hand quantities
<code>/api/v1/inventory-counts</code>	Physical count / reconciliation sessions

Reports — `/api/v1/reports`

Method	Path	Summary
GET	/reports	Overall sales report (date range)
GET	/reports/type/hourly · /type/daily · /type/monthly	Time-bucketed breakdowns
GET	/reports/product-group · /product	Revenue by category / by item
GET	/reports/user · /payment	By cashier / by payment method
GET	/reports/exportable	Aggregated unexported transactions for a date
PATCH	/reports/mark-exported	Mark that date's transactions exported
GET	/reports/cashier-x-reading	Current cashier X-Reading (today, self, own terminal)
POST	/reports/cashier-x-reading/close	Close the X-Reading into history
GET	/reports/cashier-x-reading/history[:id]	List / get past X-Readings
GET	/reports/cashier-daily-report	Current cashier daily report
POST	/reports/cashier-daily-report/close	Close it into history
GET	/reports/cashier-daily-report/history[:id]	List / get past daily reports
GET	/reports/z-reading	Current store-wide Z-Reading
POST	/reports/z-reading/close	Close (authorised) — advances the Z counter
GET	/reports/z-reading/history[:id]	List / get past Z-Readings (store-wide)

9. Health, ERP sync & cron

10.1 Health

Endpoint	Checks
GET /api/v1/health	memory + disk + postgres
GET /api/v1/health/live	memory only (liveness)
GET /api/v1/health/ready	postgres (readiness)

GET /health/live **without** the `api/v1` prefix returns 404 — expected.

10.2 ERP sync — `/api/v1/erp-sync`

Method	Path	Summary
POST	<code>/erp-sync/menu</code>	Pull the menu from the ERP into the POS
POST	<code>/erp-sync/reports/backfill</code>	Re-push historical reports to the ERP
GET	<code>/erp-sync/status</code>	Last sync status

Configured by the `ERP_*` env vars. Disabled entirely with `ERP_SYNC_ENABLED=false`.

10.3 Cron jobs

`CronJobsModule` runs scheduled work — e.g. syncing ERP-managed inventory stock every 2 minutes (`*/2 * * * *`) for products that carry an SKU.

10. Device transfer internals

`src/device-transfer/` . Endpoints (admin/supervisor only):

Method	Path	Summary
POST	<code>/api/v1/device-transfer/export</code>	Export the full <code>public</code> schema as one gzipped-then-AES-256-GCM-encrypted <code>.posbackup</code>
POST	<code>/api/v1/device-transfer/import</code>	Replace the full dataset from a <code>.posbackup</code> (destructive)

- Export includes every base table plus the `migrations` table.
- Import runs in one transaction: `SET LOCAL session_replication_role='replica'` (needs a superuser DB role — true for the bundled Postgres), `TRUNCATE ... CASCADE`, bulk re-insert preserving primary keys, then reset every sequence (parsed from `column_default`, so standalone sequences like `z_readings_z_counter_seq` are covered).
- A **strict** import is refused unless the archive's migration history is identical to the target's.
- `partialRestore: 'true'` skips the migration gate, keeps the target's own `migrations` table, and imports only tables/columns present on both devices; a table is dropped whole if the target added a required column the archive lacks, a column is dropped if its Postgres `udt` changed. Everything skipped is returned in `DeviceImportSummaryDto.skipped ( { tables, columns } )`.

NOTE See `docs/runbooks/kiosk-device-migration.md` for the operational procedure.

11. Migrations & schema

- 1 Change an entity.
- 2 `npm run migration:generate` (diff-based) or `npm run migration:create` (blank), name it with a timestamp.
- 3 Review the generated SQL. Migrations are the **only** way the schema changes.
- 4 `npm run migration:up` to apply; `npm run migration:show` to check status; `npm run migration:down` reverts the last one.
- 5 `npm run migration:sync-index` to refresh `migrations/index.ts` (`build:sea` does this automatically).

Notable migrations

Migration	Effect
1779582000000	<code>products.image_url</code> BYTEA→TEXT; adds <code>price</code> , <code>is_available</code> , <code>sort_order</code> , <code>category</code> ; creates <code>product_modifier_groups</code> junction
1779582100000	Drops the <code>catalog_</code> -prefixed tables; kiosk now reads via <code>/catalog/products</code> + <code>/products/:id</code>

The installer runs all migrations (79 at time of writing) on first install and again via `POSBackend.exe --migrate` during recovery.

12. Production build & Windows installer

13.1 Backend SEA

```
cd be
npm install
npm run build:sea      # sync indexes -> nest build -> @vercel/ncc bundle
                        # -> SEA blob -> inject into node -> be\POSBackend.exe
```

Output `be\POSBackend.exe` (~80–120 MB). Build machine needs **Node 22 LTS**.

13.2 Full installer

```
# repo root, prompts for version:
.\build-installer.bat
# 1 verify prereqs (Inno Setup 6, C:\pgsql, C:\nssm, be\.env.prod)
# 2 optionally bump #define MyAppVersion in installer.iss
# 3 flutter codegen (pub get + build_runner)
# 4 parallel: npm run build:sea + fvm flutter build windows
# 5 auto-strip curly quotes from installer.iss
# 6 compile with ISCC.exe
# -> be\installer\output\POSKiosk-Setup-<version>.exe
```

13.3 What the installer does on the target

Step	Action
0	Silently installs VC++ 2015–2022 redistributable
1	Extracts to <code>C:\POSKiosk\</code>
2–5	Initiates PostgreSQL data dir <code>C:\posdata\</code> , configures <code>postgresql.conf</code> , grants <code>NT AUTHORITY\NetworkService</code> , registers + starts <code>POSPostgres</code>
6–7	Waits for Postgres, creates <code>pos_db</code>
8–9	Runs all TypeORM migrations; optionally seeds
10	Installs <code>POSBackendService</code> via NSSM (auto-start)
11	Writes <code>C:\POSKiosk\settings.txt</code> with the wizard's kiosk number

IMPORTANT Install paths contain no spaces on purpose — `pg_ctl` cannot register a service whose binary path has spaces. Never edit `installer.iss` in an editor with smart-quote autocorrect; use `npm run check:quotes / fix:quotes`.

13. Verification & troubleshooting

Verify an install (elevated PowerShell on the target)

```
Get-Service POSPostgres, POSBackendService | Format-Table Name, Status
& "C:\POSKiosk\pgsql\bin\pg_isready.exe" -h 127.0.0.1 -p 5432
Invoke-WebRequest http://localhost:3000/api/v1/health -UseBasicParsing # 401 = endpoint up, auth active
Get-Content "C:\POSKiosk\settings.txt" # kiosk.no=<n>
```

Common issues

Symptom	Fix
App: "no internet connection"	Check both services Running; <code>pg_isready</code> ; tail <code>setup-postgres-install.log</code> and <code>backend-error.log</code> .
Internal server error on payment / sales order	<code>settings.txt</code> missing — recreate <code>kiosk.no=1</code> (ASCII) and <code>Restart-Service POSBackendService</code> .
Missing tables / <code>42P01</code>	Migrations didn't run: <code>Set-Location C:\POSKiosk\backend; .\POSBackend.exe --migrate; Restart-Service POSBackendService</code> .
<code>POSPostgres</code> not registered	Old install path had spaces — reinstall with the current installer (<code>C:\POSKiosk</code>).
<code>CreateProcess</code> failed; code <code>2</code> during install	Curly quotes in <code>installer.iss</code> <code>[Run]</code> / <code>[Files]</code> — run <code>npm run fix:quotes</code> and recompile.

Log locations

Log	Contents
<code>C:\POSKiosk\logs\setup-postgres-install.log</code>	DB init, service registration, migrations
<code>C:\POSKiosk\logs\install-backend-service-install.log</code>	NSSM service install + health check
<code>C:\POSKiosk\logs\backend-output.log</code> / <code>backend-error.log</code>	Live NestJS stdout / stderr
<code>C:\posdata\log\postgresql.log</code>	PostgreSQL server log